پردازه‌های مختلف تخصیص بدهیم؟ اگر تعداد ۹۳ قاب آزاد و دو پردازه داشته باشیم، هر یک از این پردازه‌ها چه تعداد قاب بدست می‌آورند؟

یک حالت ساده از یک سیستم را با ۱۲۸ قاب در نظر بگیرید. سیستم‌عامل، ۳۵ قاب را برداشته و ۹۳ قاب باقی را برای پردازه‌ی کاربر باقی می‌گذارد. در صفحه‌بندی نیازی محض، در ابتدا تمامی ۹۳ قاب در لیست قاب آزاد قرار می‌گیرند. لحظه‌ای که پردازه‌ای از کاربر اجرایش را شروع می‌کند، دنباله‌ای از خطاهای صفحه را تولید می‌کند. ۹۳ خطای اول صفحات، همگی قاب مورد نیاز خود را از لیست قاب آزاد بدست می‌آورند. زمانی که لیست قاب آزاد تمام شد، یک الگوریتم جایگزینی صفحه برای گزینش یکی از ۹۳ صفحه‌ی درون حافظه استفاده می‌شود تا با صفحه‌ی ۹۴ام جایگزین نماید و الی آخر. با خاتمه‌یافتن پردازه، تمامی ۹۳ قاب باز هم در لیست قاب آزاد گذاشته می‌شوند.

گونه‌های زیادی روی این راهبرد ساده وجود دارند. می‌توانیم سیستم‌عامل را مجبور کنیم که تمامی بافرهای خود و فضای جدول را از لیست قاب آزاد بر دارد. زمانی که این فضا توسط سیستم‌عامل استفاده نمی‌شود، می‌توان آن را برای پشتیبانی صفحه‌بندی کاربر به کار گرفت. می‌توانیم به تعداد سه قاب از لیست قاب آزاد را برای تمامی زمان‌ها حفظ کنیم. بنابراین، لحظه‌ای که یک خطای صفحه رخ می‌دهد، قاب آزادی برای آوردن صفحه در اختیار خواهیم داشت. در حالی که صفحه مبادله می‌شود، می‌توان جایگزینی انتخاب کرد تا در ادامه، زمانی که پردازه‌ی کاربر اجرای خود را تداوم می‌بخشد، در حافظه‌ی ثانویه نوشته شود. گونه‌های دیگری هم ممکن است، اما راهبرد اساسی روشن است: هر یک از قاب‌های آزاد را می‌توان به پردازه‌ی کاربر تخصیص داد.

۱۰-۵-۱ تعداد کمینه‌ی قاب‌ها

راهبردهای ما در تخصیص قاب‌ها به صور مختلف محدود می‌شوند. برای مثال، نمی‌توانیم تخصیصی بیشتر از تعداد کل قاب‌های موجود انجام بدهیم (مگر این که اشتراک صفحه داشته باشیم). همین‌طور بایستی تعداد حداقل قاب تخصیصی را رعایت کنیم. در اینجا، روی نیاز دوم متمرکز می‌شویم.

یکی از دلایل تخصیص حداقل قاب‌های تخصیصی کمینه به کارایی مربوط می‌شود. پر واضح است که هر چه تعداد قاب‌های تخصیصی به پردازه‌ها کاهش یابد، نرخ خطای صفحه افزایش یافته، اجرای پردازه کند می‌شود. به علاوه، به خاطر دارید که اگر خطای صفحه پیش از تکمیل اجرای یک دستورالعمل اتفاق بیفتد، بایستی اجرای آن دستورالعمل تجدید شود. در نتیجه، بایستی تعداد حداقل قابی را که یک دستورالعمل منفرد می‌تواند در اجرای خود مراجعه کند، در اختیار داشته باشیم.

برای مثال، ماشینی را در نظر بگیرید که در آن تمامی دستورالعمل‌هایی که به حافظه مراجعه دارند تنها به یک آدرس حافظه مراجعه داشته باشند. در این حالت، حداقل به یک قاب برای دستورالعمل و یک قاب هم برای مراجعه‌ی حافظه نیاز داریم. به علاوه، اگر آدرس‌دهی غیر مستقیم تا یک سطح مجاز باشد (برای مثال، دستورالعمل load که در قاب ۱۶ واقع است می‌تواند

به آدرسی از قاب ۰ مراجعه کند که آن یک مراجعه‌ی غیرمستقیم به قاب ۲۳ می‌باشد)، در این صورت صفحه‌بندی هر پردازه، حداقل به سه قاب نیاز دارد. (به این فکر کنید که اگر پردازه‌ای فقط دو قاب در اختیار داشته باشد چه اتفاقی ممکن است بیفتد.)

تعداد قاب‌های کمینه توسط معماری کامپیوتر تعریف می‌شود. برای مثال، اگر دستورالعمل انتقال یک معماری مشخصی برای برخی مدهای آدرس‌دهی بیش از یک کلمه را شامل شود، این امکان وجود دارد که خود دستورالعمل در دو قاب بیفتد. به علاوه، اگر هر یک از دو عملوند آن بتوانند مراجعات غیرمستقیم داشته باشند، درمجموع حداقل شش قاب لازم است. به‌عنوان مثالی دیگر، دستورالعمل انتقال معماری‌های اینتل ۳۲ و ۶۴ بیتی اجازه می‌دهد داده‌ها تنها از یک ثبات به ثبات و میان ثبات‌ها و حافظه جابجا شوند؛ جابجایی مستقیم حافظه-به-حافظه را اجازه نمی‌دهد، با این کار، تعداد حداقل قاب لازم برای یک پردازه محدود می‌شود.

در حالی که، تعداد قاب‌های کمینه‌ی هر یک از پردازه‌ها توسط معماری تعریف می‌شود، تعداد حداکثر آن بر اساس حجم حافظه‌ی فیزیکی موجود تعریف می‌شود. در این میان، هنوز هم انتخاب مهم در تخصیص قاب‌ها برای ما باقی می‌ماند.

۱۰-۵-۲ الگوریتم‌های تخصیص

آسان‌ترین راه تقسیم m قاب در میان n پردازه، اعطای سطم مساوی m/n قاب در میان آنهاست (با چشم‌پوشی لحظه‌ای از قاب‌های مورد نیاز سیستم‌عامل). برای نمونه، اگر تعداد ۹۳ قاب و ۵ پردازه داشته باشیم، هر یک از پردازه‌ها ۱۸ قاب در اختیار خواهند گرفت. ۳ قاب باقیمانده را نیز می‌توان برای انبار بافر قاب آزاد در نظر گرفت. این طرح، **تخصیص برابر** نام دارد.

روش دیگر آن است که حجم متفاوت مورد نیاز حافظه‌ی پردازه‌های مختلف را تشخیص بدهیم. سیستمی را با اندازه‌ی قاب‌های ۱ کیلوبایتی در نظر بگیرید. اگر یک پردازه‌ی کوچک دانشجویی ۱۰ کیلوبایتی و یک پایگاه‌داده‌ی محاوره‌ای ۱۲۷ کیلوبایتی تنها دو پردازه‌ای باشند که می‌خواهند در یک سیستم با ۶۲ قاب آزاد اجرا شوند، چندان عاقلانه نیست که به هر یک از این پردازه‌ها ۳۱ قاب تخصیص بدهیم. پردازه‌ی دانشجویی بیشتر از ۱۰ قاب را نیاز ندارد، لذا به بیان بسیار ساده، ۲۱ قاب دیگر هدر می‌روند.

برای حل این مسأله می‌توانیم از **تخصیص متناسب** استفاده کنیم که در آن، حافظه‌ی موجود مطابق با اندازه‌های هر یک از پردازه‌ها تخصیص می‌یابد. اگر s_i اندازه‌ی حافظه‌ی مجازی پردازه‌ی p_i باشد، در این صورت کل فضای حافظه‌ی مجازی پردازه‌ها به صورت زیر تعریف می‌شود:

$$S = \sum s_i$$

در این صورت، اگر تعداد کل قاب‌های موجود m باشد، a_i قاب برای پردازه‌ی p_i تخصیص می‌دهیم که مقدار تقریبی آن برابر است با:

$$a_i = s_i / S \times m$$

البته، بایستی مقدار صحیح هر یک از a_iها را طوری انتخاب کنیم که از تعداد قاب‌های کمینه‌ی

مورد نیاز مجموعه‌ی دستورالعمل بزرگ‌تر باشد، ضمن این که نبایستی مجموع تعداد قاب‌های تخصیصی پردازه‌ها از m تجاوز نماید.

با تخصیص متناسب، تعداد ۶۲ قاب موجود را میان دو پردازه که یکی ۱۰ صفحه و دیگری ۱۲۷ صفحه می‌باشند، به ترتیب با تخصیص ۴ و ۵۷ قاب تقسیم می‌کنیم چرا که:

$$۱۰/۱۳۷ \times ۶۲ \approx ۴ \text{ و}$$

$$۱۲۷/۱۳۷ \times ۶۲ \approx ۵۷.$$

با این روش، هر یک از پردازه‌ها از قاب‌ها به جای سهم مساوی، مطابق با «نیازشان» سهم می‌برند.

البته، تخصیص هر دو روش تخصیص مساوی و متناسب ممکن است مطابق سطح چندبرنامگی تغییر یابد. اگر سطح چندبرنامگی افزایش داده شود، هر یک از پردازه‌ها برخی قاب‌های خود را از دست می‌دهند تا حافظه‌ی کافی مورد نیاز پردازه‌ی جدید فراهم شود. برعکس، اگر سطح چندبرنامگی کاهش یابد، قاب‌های حاصل از پردازه‌ای که تمام شده است می‌تواند در میان پردازه‌های باقیمانده توزیع شود.

توجه کنید که با هر دو روش تخصیص مساوی و یا متناسب، با پردازه‌ای که الویت بالایی دارد، همانند الویت پایین رفتار می‌شود. با این وجود، مطابق تعریف ممکن است بخواهیم جهت تسریع اجرای پردازه‌ای با الویت بالا، حافظه‌ی بیشتری در اختیار آن قرار بدهیم که در این میان، پردازه‌ای که الویت پایینی دارد زیان خواهد دید. یک راه‌حل آن است که از طرح تخصیص متناسب استفاده کنیم، اما ضریب تخصیص قاب‌ها را به جای اندازه‌ی پردازه‌ها، الویت پردازه‌ها یا ترکیبی از اندازه و الویت را لحاظ کنیم.

۱۰-۵-۳ تخصیص سراسری در مقایسه با محلی

عامل مهم دیگر مربوط به تخصیص قاب‌های پردازه‌های مختلف، جایگزینی صفحه است. می‌توانیم الگوریتم‌های جایگزینی صفحه را با داشتن پردازه‌های متعددی که برای کسب قاب‌ها رقابت می‌کنند در دو دسته‌ی وسیع رده‌بندی کنیم: **جایگزینی سراسری** و **جایگزینی محلی**. جایگزینی سراسری به پردازه اجازه می‌دهد تا قاب جایگزینی را از مجموعه‌ی تمامی قاب‌ها انتخاب نماید، حتی اگر فعلاً آن قاب در اختیار پردازه‌ای دیگر باشد؛ یعنی، پردازه می‌تواند قاب پردازه‌ی دیگری را بردارد. جایگزینی محلی لازم دارد که هر یک از پردازه‌ها فقط از مجموعه‌ی قاب‌های تخصیصی خودشان انتخاب داشته باشند.

برای مثال، طرح تخصیصی را در نظر بگیرید که به پردازه‌های با الویت بالا اجازه می‌دهیم که بتوانند برای جایگزینی از قاب‌های پردازه‌های با الویت پایین نیز انتخاب داشته باشند. پردازه می‌تواند یک جایگزین را از میان قاب‌های در اختیار خود یا قاب‌های هر یک از پردازه‌های با الویت پایین انتخاب کند. این رویکرد به پردازه‌ای که الویت بالایی دارد اجازه می‌دهد که قاب‌های تخصیصی خود را با هزینه‌ی پردازه‌ی الویت پایین افزایش بدهد. در حالی که در راهبرد جایگزینی محلی، تعداد قاب‌های تخصیصی هر یک از پردازه‌ها تغییر نمی‌یابد. با جایگزینی سراسری ممکن

است شرایطی پیش بیاید که یک پردازه، جایگزینی را فقط از قاب‌های تخصیصی سایر پردازه‌ها انجام بدهد، لذا تعداد قاب‌های تخصیصی آن افزایش می‌یابد (با این فرض که، سایر پردازه‌ها قاب‌های آن را برای جایگزینی انتخاب نکنند).

یک مسأله در رابطه با الگوریتم جایگزینی سراسری این است که یک پردازه نمی‌تواند نرخ خطای صفحه‌ی خاص خود را کنترل کند. مجموعه‌ی صفحات پردازه که داخل حافظه قرار دارند تنها به رفتار صفحه‌بندی خود پردازه مربوط نمی‌شود بلکه به رفتار صفحه‌بندی سایر پردازه‌ها نیز بستگی دارد. بنابراین، یک پردازه ممکن است به خاطر شرایطی خارج از اختیار خود رفتار کاملاً متفاوتی داشته باشد (برای مثال، در یک اجرا ۰/۵ ثانیه طول بکشد و در اجرایی دیگر ۴/۳ ثانیه زمان ببرد). این مورد در الگوریتم جایگزینی محلی پیش نمی‌آید. با جایگزینی محلی، مجموعه‌ی صفحاتی از پردازه که در داخل حافظه قرار دارد تنها از رفتار صفحه‌بندی همان پردازه متأثر خواهد شد. جایگزینی محلی به پردازه اجازه نمی‌دهد که صفحات سایر پردازه‌ها را هرچند که کمتر هم استفاده داشته باشند در اختیار بگیرد. بنابراین، جایگزینی سراسری عموماً بازده بالایی را برای سیستم به بار می‌آورد. لذا، این روش رواج بیشتری دارد.

خطاهای صفحه‌ی اصلی و فرعی

همان‌گونه که در بخش ۱۰-۲-۱ بیان شد، خطای صفحه زمانی اتفاق می‌افتد که یک صفحه فاقد یک نگاشت معتبر در فضای آدرس یک پردازه است. سیستم‌عامل‌ها عموماً میان دو نوع خطای صفحه تمایز قایل می‌شوند: **خطاهای اصلی**[1] و **فرعی**[2]. (ویندوز به خطاهای اصلی و فرعی، به‌ترتیب **خطاهای سخت** و **نرم** اطلاق می‌کند.) خطای اصلی زمانی اتفاق می‌افتد که صفحه‌ای مراجعه می‌شود و صفحه در حافظه نیست. سرویس‌دهی به یک خطای صفحه مستلزم خواندن صفحه‌ی مطلوب از ذخیره‌ی پشتیبان به یک قاب آزاد و به‌روزرسانی جدول صفحه است. صفحه‌بندی نیازی نوعاً یک نرخ خطای صفحه‌ی اصلی بالایی در آغاز تولید می‌کند.

خطای صفحه‌ی فرعی زمانی اتفاق می‌افتد که یک پردازه فاقد یک نگاشت منطقی به یک صفحه است، در حالی که صفحه در حافظه می‌باشد. خطاهای فرعی می‌توانند با یکی از این دو دلیل اتفاق بیفتند. اولاً، یک پردازه ممکن است به یک کتابخانه‌ی مشترکی مراجعه کند که در حافظه است، اما پردازه در جدول صفحه‌اش نگاشتی به آن ندارد. در اینجا، تنها لازم است که جدول صفحه به‌روز شود تا به صفحه‌ی موجود در حافظه ارجاع دهد. علت دوم وقوع خطاهای فرعی زمانی است که یک صفحه از یک پردازه پس‌گرفته شده و در لیست قاب آزاد گذاشته می‌شود، اما صفحه هنوز صفر نشده و به پردازه‌ی دیگری تخصیص نیافته است. موقعی که خطای صفحه‌ی این نوعی پیش می‌آید، قاب از لیست قاب آزاد برداشته شده و دومرتبه به پردازه گمارده می‌شود. همان‌گونه که انتظار می‌رود، رفع خطای صفحه‌ی فرعی نوعاً به نسبت رفع خطای صفحه‌ی اصلی، زمان بسیار کمتری می‌برد.

شما می‌توانید تعداد خطاهای صفحه‌ی اصلی و فرعی را در یک سیستم لینوکس با استفاده از فرمان ps – eo min_flt, maj_flt, cmd مشاهده کنید که تعداد خطاهای صفحه‌ی اصلی و فرعی، همین‌طور فرمانی که پردازه صادر کرده را خروجی می‌نماید. یک نمونه‌ی خروجی از این فرمان ps به‌صورت زیر است:

1. Major
2. Minor

```
MINFL           MAJFL           CMD
186509          32              /usr/lib/systemd/systemd-logind
76822           9               /usr/sbin/sshd -D
1937            0               vim 10.tex
699             14              /sbin/auditd -n
```

در اینجا جالب است که توجه کنید در اکثر فرمان‌ها، تعداد خطاهای صفحه‌ی اصلی عموماً خیلی پایین است، در حالی که تعداد خطاهای صفحه‌ی فرعی بسیار بالاتر است. این موضوع نشان می‌دهد که پردازه‌های لینوکس از مزیت کتابخانه‌های مشترک به‌صورت قابل توجهی بهره می‌برند، همین که یک کتابخانه به‌داخل حافظه بارگذاری می‌شود، خطاهای بعدی فقط خطاهای فرعی هستند.

در ادامه، ما بر روی یک راهبرد ممکن که می‌توانیم برای پیاده‌سازی یک سیاست جایگزینی صفحه‌ی سراسری استفاده کنیم تمرکز می‌کنیم. با این رویکرد، تمامی درخواست‌های حافظه را از لیست قاب آزاد ارضا می‌کنیم، اما به‌جای انتظارکشی این که لیست به صفر برسد قبل از این که صفحاتی را برای جایگزینی انتخاب کنیم، ما جایگزینی صفحه را موقعی که لیست به زیر آستانه‌ی معینی می‌افتد راه می‌اندازیم. این راهبرد در تلاش است تا تضمین کند همواره حافظه‌ی آزاد کافی برای ارضای درخواست‌های جدید وجود دارد.

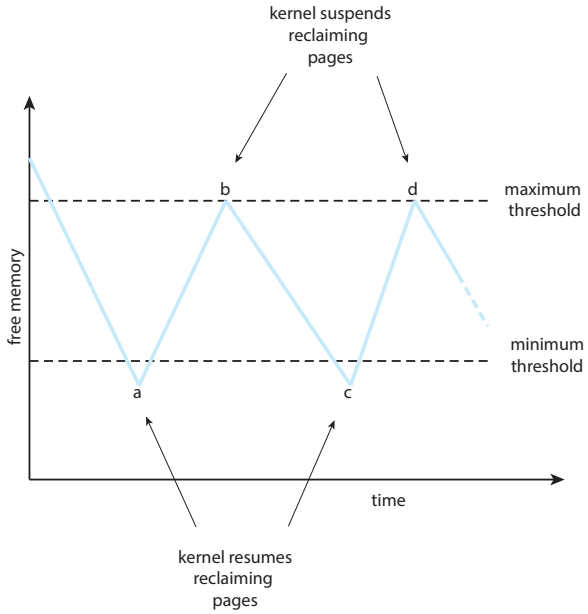
یک چنین راهبردی در شکل ۱۰–۱۸ به‌تصویر کشیده می‌شود. هدف این راهبرد، نگه‌داشتن میزان حافظه‌ی آزاد، بالای یک آستانه‌ی حداقلی است. موقعی که به پایین‌تر از این آستانه می‌افتد، یک روال هسته راه‌اندازی می‌شود که شروع می‌کند به پس‌گیری صفحات از تمامی پردازه‌های سیستم (نوعاً به‌استثنای هسته). این قبیل روال‌های هسته غالباً به **دروگر**[1] معروف هستند و می‌توانند هر کدام از الگوریتم‌های جایگزینی صفحه‌ی پوشش داده شده در بخش ۱۰–۴ را به‌کار بگیرند. موقعی که میزان حافظه‌ی آزاد به آستانه‌ی حداکثر می‌رسد، روال دروگر معلق می‌شود، تنها در صورتی از سر گرفته می‌شود که حافظه‌ی آزاد دوباره به زیر آستانه‌ی حداقلی می‌افتد.

در شکل ۱۰–۱۸ می‌بینیم، میزان حافظه‌ی آزاد در نقطه‌ی a به زیر آستانه‌ی حداقلی می‌افتد و هسته پس‌گیری صفحات و اضافه‌کردن آن‌ها به لیست قاب آزاد را شروع می‌کند. این کار ادامه دارد تا این که به آستانه‌ی حداکثر برسد (نقطه‌ی b). به مرور زمان، درخواست‌های دیگری برای حافظه وجود دارد، و در نقطه‌ی c، میزان حافظه‌ی آزاد باز هم به زیر آستانه‌ی حداقل می‌افتد. پس‌گیری صفحه از سر گرفته می‌شود، تنها زمانی معلق می‌شود که میزان حافظه‌ی آزاد به آستانه‌ی حداکثری برسد (نقطه‌ی d). این فرایند ادامه می‌یابد مادامی که سیستم در حال اجراست.

همان‌گونه که در بالا عنوان شد، روال دروگر هسته می‌تواند هر کدام از الگوریتم‌های جایگزینی صفحه را بر دارد، اما نوعاً شکلی از تقریب LRU را استفاده می‌کند. در نظر بگیرید که چه اتفاق می‌افتد در صورتی که روال دروگر نتواند لیست قاب آزاد را در حداقل آستانه نگه بدارد. تحت این شرایط، روال دروگر ممکن است پس‌گیری صفحات را با خشونت بیشتری انجام دهد. برای مثال، شاید الگوریتم شانس دوم را کنار گذاشته و از FIFO محض استفاده نماید. مثال حدی دیگری در لینوکس اتفاق می‌افتد؛ موقعی که میزان حافظه‌ی آزاد به سطوح بسیار پایینی می‌رسد،

1. Reaper

روالی تحت عنوان **قاتل کمبود حافظه**[1] **(OOM)** پردازه‌ای را برای خاتمه‌دادن انتخاب کرده و با این کار، حافظه‌ی آن را آزاد می‌کند. لینوکس از کجا تعیین می‌کند که کدام پردازه را خاتمه دهد؟ هر پردازه چیزی تحت عنوان نمره‌ی OOM دارد، نمره‌ی بالا احتمال این که پردازه توسط روال قاتل OOM خاتمه داده شود را افزایش می‌دهد. نمرات OOM مطابق درصد حافظه‌ای که یک پردازه استفاده می‌کند محاسبه می‌شوند—درصد بالاتر، نمره‌ی OOM بالاتر. (نمرات OOM را می‌توان در سیستم‌فایل proc/ مشاهده کرد که نمره‌ی یک پردازه با pid 2500 می‌تواند به‌صورت proc/2500/oom_score/ دیده شود.)



شکل ۱۰-۱۸ پس‌گیری صفحات.

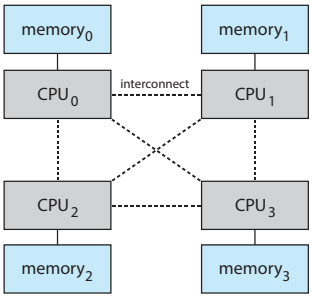
در حالت کلی، نه تنها روال‌های دروگر می‌توانند بر اساس میزان خشونتی که در پس‌گیری حافظه به خرج می‌دهند فرق داشته باشند، بلکه مقادیر کمینه و بیشینه‌ی آستانه‌ها نیز می‌تواند فرق داشته باشد. این مقادیر می‌تواند به مقادیر پیش‌فرض مقداردهی شوند، اما برخی سیستم‌ها ممکن است به یک مدیر سیستم اجازه دهند تا آن‌ها را بر اساس میزان حافظه‌ی فیزیکی موجود در سیستم پیکربندی نماید.

۱۰-۵-۴ دسترسی حافظه‌ی غیریکپارچه

در مباحثی که تاکنون از حافظه‌ی مجازی داشتیم، فرض بر این بود که تمامی حافظه‌ی اصلی به صورت یکسان ساخته می‌شود —یا حداقل زمان دسترسی مساوی دارد. در سیستم‌های

1. Out-of-memory (OOM)

دسترسی حافظه‌ی غیریکپارچه[1] **(NUMA)** با پردازنده‌های متعدد (بخش ۱-۳-۲) این‌گونه نیست. در این سیستم‌ها، یک پردازنده‌ی مشخص می‌تواند برخی از بخش‌های حافظه‌ی اصلی را سریع‌تر از سایر بخش‌ها دسترسی نماید. این تفاوت کارایی به چگونگی اتصالات پردازنده‌ها و حافظه در سیستم بر می‌گردد. چنین سیستمی از پردازنده‌های متعددی ساخته می‌شود که هر یک حافظه‌ی محلی خاص خود را دارند (شکل ۱۰-۱۹). پردازنده‌ها با استفاده از یک اتصال سیستمی مشترک سازمان‌دهی می‌شوند، و همان‌گونه انتظار دارید، یک پردازنده می‌تواند حافظه‌ی محلی خود را سریع‌تر از حافظه‌ی محلی پردازنده‌ای دیگر دسترسی نماید. سیستم‌های NUMA بدون استثناء کندتر از سیستم‌هایی هستند که در آن‌ها با تمامی دسترسی‌ها به حافظه‌ی اصلی یکسان رفتار می‌شود. با این وجود، همان‌گونه که در بخش ۱-۳-۲ بیان شد، سیستم‌های NUMA می‌توانند پردازنده‌های بیشتری را پوشش داده و از این رو به‌سطوح بالاتری از بازده و موازی‌سازی دست یابند.



شکل ۱۰-۱۹ معماری چندپردازشی NUMA.

مدیریت این که کدام یک از قاب‌صفحات در کدام مکان‌ها ذخیره شده‌اند می‌تواند کارایی سیستم‌های NUMA را به صورت چشم‌گیری تحت تأثیر قرار بدهد. اگر حافظه را در چنین سیستمی به صورت یکپارچه در نظر بگیریم، پردازنده‌ها ممکن است انتظار طولانی‌تر درخوری در دسترسی حافظه داشته باشند نسبت به این که الگوریتم‌های تخصیص حافظه را با احتساب NUMA تغییر بدهیم. ما برخی از این اصلاحات را در بخش ۵-۵-۴ بیان کردیم. هدف آن‌ها این است که تخصیص قاب‌های حافظه را تا جایی که ممکن است نزدیک پردازنده‌ای که پردازه بر روی آن اجرا می‌شود انجام بدهیم. (تعریف *نزدیکی* همان «تأخیر کمینه» است که نوعاً به معنی همان برد سیستم است که پردازنده قرار دارد.) بنابراین، موقعی که پردازه‌ای گرفتار یک خطای صفحه می‌شود، سیستم حافظه‌ی مجازی آگاه به NUMA، قابی را به آن پردازه تخصیص می‌دهد که حتی‌الامکان نزدیک به پردازنده‌ای باشد که پردازه در آن اجرا دارد.

زمانبند به‌منظور لحاظ NUMA بایستی پیگیر آخرین پردازنده‌ای باشد که پردازه بر روی آن اجرا داشته است. اگر زمانبند تلاش نماید که هر یک از پردازه‌ها را بر روی پردازنده‌ی قبلی خودشان زمانبندی کند و سیستم حافظه‌ی مجازی هم سعی کند قاب‌های تخصیصی پردازه

1. Non-uniform memory access (NUMA)

نزدیک پردازنده‌ای باشند که برای اجرا زمانبندی می‌شود، در این صورت توفیق بیشتر کَش و تقلیل زمان‌های دسترسی حافظه قابل حصول است.

اوضاع با آمدن نخ‌ها از این هم پیچیده‌تر می‌شود. برای مثال، یک پردازه با نخ‌های اجرایی فراوان به جایی می‌رسد که نخ‌های آن بر روی بردهای چندین سیستم متفاوت زمانبندی می‌شوند. در این حالت، تخصیص حافظه بایستی چگونه صورت بگیرد؟

همان‌گونه که در بخش ۵-۷-۱ بحث کردیم، لینوکس این وضعیت را با اجبار هسته به شناسایی یک سلسله‌مراتبی از دامنه‌های زمان‌بندی مدیریت می‌کند. زمان‌بند CFS لینوکس به نخ‌ها اجازه نمی‌دهد تا در میان دامنه‌های متفاوت مهاجرت داشته و لذا گرفتار جریمه‌های دسترسی شوند. لینوکس برای هر گره NUMA، یک لیست قاب آزاد جداگانه‌ای دارد، با این کار تضمین است که تخصیص حافظه‌ی یک نخ از گرهی صورت می‌گیرد که روی آن اجرا می‌شود. با تشابهی، سولاریس این مسأله را با ایجاد **lgroups** (اختصار «گروه‌های همسایگی[1]») در هسته حل می‌کند. هر lgroup پردازنده‌ها و حافظه را گرد هم می‌آورد و هر پردازنده در گروه می‌تواند هر حافظه در گروه را در محدوده‌ی یک فاصله‌ی تأخیر تعریف شده دسترسی نماید. به‌علاوه، یک سلسله‌مراتب از lgroupها مطابق میزان دیرکرد میان گروه‌ها، شبیه به سلسله‌مراتب دامنه‌های زمان‌بندی در لینوکس، وجود دارد. سولاریس تلاش می‌کند که تمامی نخ‌های یک پردازه را زمانبندی کرده و تمامی حافظه‌ی یک پردازه را از یک lgroup تخصیص بدهد. اگر این کار مقدور نباشد، برای باقی منابع مورد نیاز، lgroupهای نزدیک را بر می‌دارد. این موضوع، دیرکرد کلی حافظه را کمینه کرده و نرخ توفیق کَش پردازنده را بیشینه می‌سازد.

۱۰-۶ کوبیدگی

در نظر بگیرید که چه اتفاقی می‌افتد اگر یک پردازه قاب «کافی» نداشته باشد—یعنی، تعداد حداقل قابی که نیاز دارد تا مجموعه‌ی کاری خود را پشتیبانی کند در اختیار ندارد. پردازه خیلی سریع خطای صفحه می‌دهد. در اینجا، دنبال جایگزینی برای صفحه می‌گردد. اما، چون تمامی صفحات آن استفاده‌ی اکیدی دارند، بایستی صفحه‌ای را جایگزین کند که دوباره چند لحظه بعد استفاده خواهد داشت. در نتیجه، باز هم به سرعت خطای صفحه خواهد داد و به دفعات این قصه تکرار می‌شود، صفحاتی را برای جایگزینی انتخاب می‌کند که بلافاصله دوباره به حافظه می‌آورد.

این فعالیت شدید صفحه‌بندی، **کوبیدگی**[2] نام دارد. پردازه‌ای کوبیده می‌شود که بیشتر زمان خود را به جای اجرا صرف صفحه‌بندی نماید. همان‌گونه که انتظار دارید کوبیدگی منجر به مسئله‌های جدی کارایی می‌شود.

۱۰-۶-۱ علت کوبیدگی

شرایط زیر را در نظر بگیرید که برگرفته از رفتار واقعی سیستم‌های صفحه‌بندی اولیه است.

1. Locality groups
2. Thrashing